

2. Conceptos de Comunicación Cliente-Servidor

Cuando hablamos de AJAX, siempre estamos hablando de **intercambio de información entre el cliente (el navegador) y el servidor**.

2.1 Petición (Request) y Respuesta (Response)

- **Request (Petición):** Es lo que envía el navegador al servidor.
Contiene:
 - Método HTTP (GET, POST, etc.)
 - URL del recurso
 - Datos enviados (opcional)
 - Cabeceras (headers) de información adicional
- **Response (Respuesta):** Es lo que devuelve el servidor.
Contiene:
 - Código de estado HTTP (200 = OK, 404 = No encontrado, 500 = Error servidor)
 - Datos devueltos (HTML, JSON, XML, texto)
 - Cabeceras de respuesta

2.2 Objetos JavaScript para AJAX

En JavaScript hay dos formas principales de realizar peticiones AJAX:

1. **XMLHttpRequest (XHR)** – Método clásico.
2. **Fetch API** – Método moderno y más sencillo.

2.2.1 XMLHttpRequest

Propiedades importantes:

- `readyState` → indica el estado de la petición (0 a 4):
 1. 0: No inicializado
 2. 1: Conectando
 3. 2: Recibido
 4. 3: Procesando
 5. 4: Completado
- `status` → código HTTP devuelto por el servidor
- `responseText` → respuesta en texto plano
- `responseXML` → respuesta en formato XML

Métodos importantes:

- `open(metodo, url, async)` → inicializa la petición
- `send(datos)` → envía la petición al servidor
- `setRequestHeader(header, valor)` → define cabeceras HTTP adicionales

2.2.2 Fetch API (moderna)

Sintaxis básica:

```
fetch("archivo.txt")
  .then(response => response.text())
  .then(data => console.log(data))
  .catch(error => console.error(error));
```

- Devuelve una **promesa**.
- Más fácil de usar que XMLHttpRequest.
- Permite manejar respuestas en distintos formatos (text, json, blob, formData).

2.3 Ejemplo práctico con XMLHttpRequest

Objetivo: Hacer una petición a un archivo JSON y mostrar los datos.

HTML

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>AJAX con JSON</title>
</head>
<body>
  <h1>Datos de usuario</h1>
  <button id="btnCargar">Cargar usuario</button>
  <pre id="resultado"></pre>

  <script src="script.js"></script>
</body>
</html>
```

JavaScript (script.js)

```
document.getElementById("btnCargar").addEventListener("click",
function() {
  let xhr = new XMLHttpRequest();
  xhr.open("GET", "usuario.json", true);
  xhr.onload = function() {
    if (xhr.status === 200) {
      let usuario = JSON.parse(xhr.responseText);
      document.getElementById("resultado").innerText =
        `Nombre: ${usuario.nombre}\nEdad:
${usuario.edad}\nEmail: ${usuario.email}`;
    } else {
      document.getElementById("resultado").innerText = "Error al
cargar los datos.";
    }
  };
  xhr.send();
});
```

```
});
```

Archivo usuario.json

```
{
  "nombre": "Ana Pérez",
  "edad": 28,
  "email": "ana.perez@mail.com"
}
```

Funcionamiento:

- Al hacer clic en el botón, se solicita el archivo `usuario.json` al servidor.
- La respuesta se convierte de JSON a objeto JavaScript con `JSON.parse()`.
- Los datos se muestran dinámicamente en el navegador sin recargar la página.

2.4 Ejemplo práctico con Fetch API

```
document.getElementById("btnCargar").addEventListener("click",
function() {
  fetch("usuario.json")
    .then(response => {
      if (!response.ok) throw new Error("Error al cargar los
datos");
      return response.json();
    })
    .then(usuario => {
      document.getElementById("resultado").innerText =
        `Nombre: ${usuario.nombre}\nEdad:
${usuario.edad}\nEmail: ${usuario.email}`;
    })
    .catch(error => {
      document.getElementById("resultado").innerText = error;
    });
});
```

2.5 Ejercicios propuestos

1. Crear un archivo JSON con información de 3 usuarios y mostrar todos los usuarios en la página usando `XMLHttpRequest`.
2. Reescribir el ejercicio anterior usando `Fetch API`.
3. Modificar el ejemplo para que, si ocurre un error (archivo no encontrado), muestre un mensaje personalizado de error.